

```

import numpy as np
import matplotlib.pyplot as plt

# Task 1: Leaky ReLU Activation Function

def leaky_relu(x, alpha=0.01):
    return np.where(x > 0, x, alpha * x)

# Test on values from -5 to 5
x = np.arange(-5, 6)

print("Input Values:")
print(x)

print("\nLeaky ReLU Output:")
print(leaky_relu(x))

# Task 2: Parametric ReLU (PReLU)

def parametric_relu(x, a):
    return np.where(x > 0, x, a * x)

def relu(x):
    return np.maximum(0, x)

x_values = np.linspace(-5, 5, 300)

plt.figure(figsize=(10,6))

plt.plot(x_values, relu(x_values),
        label="ReLU")

plt.plot(x_values, leaky_relu(x_values),
        label="Leaky ReLU (alpha=0.01)")

plt.plot(x_values, parametric_relu(x_values, 0.2),
        label="PReLU (a=0.2)")

plt.title("Comparison of ReLU, Leaky ReLU and PReLU")
plt.xlabel("Input")
plt.ylabel("Output")
plt.grid(True)
plt.legend()

```

```
plt.show()
```

Task 3: Softmax Function

```
def softmax(x):  
    exp_values = np.exp(x - np.max(x))  
    return exp_values / np.sum(exp_values)
```

```
scores = np.array([2.0, 1.0, 0.1, 0.5])
```

```
probabilities = softmax(scores)
```

```
print("\nSoftmax Probabilities:")
```

```
for p in probabilities:  
    print(round(p,3))
```

Task 4: Hard Sigmoid

```
def hard_sigmoid(x):  
    return max(0, min(1, 0.2*x + 0.5))
```

```
inputs = [-3, -1, 0, 1, 3]
```

```
print("\nHard Sigmoid Outputs")
```

```
for i in inputs:  
    print(f"x = {i} --> {hard_sigmoid(i)}")
```

Task 5: Activation Function Selection

```
print("\nActivation Function Selection")
```

```
print("(a) Hidden Dense Layer -> ReLU")  
print("Reason: Fast training, avoids vanishing gradient, computationally efficient.")
```

```
print("\n(b) Output Layer (5 Classes) -> Softmax")  
print("Reason: Converts outputs into probabilities whose sum equals 1.")
```

```
print("\n(c) Output Layer (Binary Classification) -> Sigmoid")  
print("Reason: Produces probability between 0 and 1 for two-class prediction.")
```

Explanation

Task 1: Leaky ReLU

Formula

$$\text{LeakyReLU}(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$$

where

$$\alpha = 0.01$$

Example:

Input Output

-5 -0.05

-2 -0.02

0 0

3 3

5 5

Why Leaky ReLU?

Unlike ReLU, Leaky ReLU allows a **small, non-zero output for negative inputs**, helping prevent neurons from becoming permanently inactive (the **dying ReLU** problem).

Task 2: Parametric ReLU (PReLU)

Formula

$$\text{PReLU}(x) = \begin{cases} x, & x > 0 \\ ax, & x \leq 0 \end{cases}$$

where **a** is a learnable parameter.

For this task:

$$a = 0.2$$

Graph Comparison

- **ReLU:** Negative inputs become exactly **0**.

- **Leaky ReLU:** Negative inputs have a small fixed slope (**0.01**).
 - **PReLU:** Negative slope is **learned during training**, making it more flexible.
-

Task 3: Softmax

Formula

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum e^{x_j}}$$

Input scores:

[2.0, 1.0, 0.1, 0.5]

Output (approximately):

0.575

0.211

0.086

0.128

These probabilities:

- Are all between **0 and 1**.
 - Sum to **1.0**.
 - Indicate the first class has the highest predicted probability.
-

Task 4: Hard Sigmoid

Formula

$$\text{hard_sigmoid}(x) = \max(0, \min(1, 0.2x + 0.5))$$

Example results:

Input Output

-3 0

-1 0.3

0 0.5

1 0.7

Input Output

3 1

Hard Sigmoid is a computationally efficient approximation of the standard sigmoid and is often used in lightweight neural networks.

Task 5: Best Activation Function Choices

(a) Hidden Dense Layer → ReLU

Reason: ReLU is fast to compute, reduces the vanishing gradient problem, and enables efficient training of deep neural networks.

(b) Output Layer (5 Classes) → Softmax

Reason: Softmax converts raw scores into probabilities that sum to 1, making it ideal for multi-class classification.

(c) Output Layer (Binary Classification) → Sigmoid

Reason: Sigmoid outputs a single probability between 0 and 1, which is well-suited for binary classification tasks.